A Machine-Checked No-Go Certification for the Six-Qubit Moment Orbits

Formalization notes

Abstract

Fifty-one modules of this development each carry a Lean 4 proof, checked by the Lean kernel alone: their axiom footprint is exactly `[propext, Classical.choice, Quot.sound]`, with no appeal to `native_decide` or `sorry` in any proof or tactic position. This note describes what was certified, the standard the certification was held to, the architecture of the development, the handful of algebraic mechanisms that between them close all fifty-one cases, and how to reproduce the recorded checks. It also reports, in Section 8, the reasoning errors that were made and caught during the work, because the way they were caught is the most transferable part of the exercise.

Scope, stated precisely. Each orbit carries two theorems. The first, `no_go_of_moments`, takes its Knill–Laflamme rows as hypotheses and derives a contradiction. The second, `no_code`, assumes only that the two logical states are supported on the residue-0 and residue-δ fibers, are normalized, and satisfy `SS.HasDistance ![c, d] 3`; every Knill–Laflamme row is then *derived* through `SixQubit/CodeBridge.lean`. So the development does establish, inside the kernel, that no exact $((6, 2, 3))$ code is supported on any of these fifty-one orbits. What it does *not* yet establish is that every code must be supported on one of them: the reduction of an arbitrary code to one of the sixty-six rank-compatible moment orbits, and the transport of other angle records to the chosen representatives, are not formalized. Section 11 states this boundary.

Contents

1 What is certified **2**
- 1.1 The statement . . . 2
- 1.2 The certification standard . . . 3
- 1.3 The result . . . 3
- 1.4 Toolchain . . . 4

2 Architecture **4**
- 2.1 The `SS` layer . . . 4
- 2.2 Transcription modules . . . 4
- 2.3 Algebra modules . . . 4
- 2.4 The audit module . . . 5

3 The mechanisms **5**

4 The four hard orbits **6**
- 4.1 Orbit 33: a block reflection . . . 6
- 4.2 Orbit 29: a K_4 of orthogonalities . . . 6
- 4.3 Orbit 31: a 4×4 table . . . 6
- 4.4 Orbit 26: the cross-anchor contraction . . . 7

5 Orbit 36: rigidity rather than nonexistence **7**
5.1 The statement . . . 8
5.2 What the equivalence means . . . 8
5.3 The chain . . . 8
5.4 Non-vacuity . . . 8
5.5 Orbit 36 at the level of codes . . . 9
5.6 The gate that did not exist . . . 9

6 Ten realizations **10**
6.1 One layer, ten tables . . . 10
6.2 Two independent computations of the same certificate . . . 10
6.3 What the kernel cannot check . . . 11
6.4 A cross-check worth recording . . . 11
6.5 Five exact images . . . 11
6.6 Three more enumerators, and the line the mask draws . . . 12
6.7 The shadow inequality, and what is still missing . . . 12

7 A tooling bug that gated three orbits **12**

8 Errors made, and how they were caught **13**

9 Reproduction **14**
9.1 Checking the proofs . . . 14
9.2 Regenerating the Lean . . . 15
9.3 Re-running the independent checks . . . 15
9.4 The workflow records . . . 15

10 Repository layout **16**

11 Open items **16**

1 What is certified

1.1 The statement

Fix q and an angle vector $a \in (\mathbb{Z}/q)^6$. The residue fibers are

$$S_0 = \{x \in \mathbb{F}_2^6 : \textstyle\sum_i a_i x_i \equiv 0 \bmod q\}, \qquad S_1 = \{x \in \mathbb{F}_2^6 : \textstyle\sum_i a_i x_i \equiv \delta \bmod q\},$$

and a putative code supported on this orbit is a pair of unit vectors

$$|0_L\rangle = \sum_{x \in S_0} c_x |x\rangle, \qquad |1_L\rangle = \sum_{y \in S_1} d_y |y\rangle.$$

Distance three means the Knill–Laflamme conditions hold for every Pauli P of weight at most two:

$$\langle 0_L|P|1_L\rangle = 0, \qquad \langle 0_L|P|0_L\rangle = \langle 1_L|P|1_L\rangle.$$

For each of the fifty-one orbits the development proves that this system has no solution. The Lean statement is a theorem named `no_go_of_moments` in each orbit's module, of the shape

```
theorem no_go_of_moments (c d : SS.BitString 6 -> C)
    (hm : MomentSystem (normSq (c y0)) ... (normSq (d z15)))
    (E_ : offSum c d ef_ w_ = 0) ...
    (D_ : diagBot c ef_ w_ = diagTop d ef_ w_) ...
    : False
```

Here `offSum` is the off-diagonal Knill–Laflamme sum for a flip mask and a Z-dressing. The functions `diagBot` and `diagTop` are the two sides of a diagonal condition. Finally, `MomentSystem` contains the pure-Z diagonal conditions, the two normalizations, and nonnegativity of every probability.

1.2 The certification standard

Five rules were fixed at the outset and held for all fifty-one orbits.

Rule 1. The transcription of the two fibers is certified by `decide` over all sixty-four bit strings, not asserted. Each module proves `S0_eq_fiber` and either `S1_eq_fiber` or `S1_residue`; these state that the listed support is exactly the required residue fiber.

Rule 2. Moment data are derived, not assumed. Every probability identity used in a proof is a named theorem whose sole hypothesis is `MomentSystem`. One invocation of `linear_combination`, with coefficients from an exact rational reduction against the moment matrix, closes it.

Rule 3. Most transcription equations were generated by script from verified matrices. For those generated modules, incidence lists, Walsh sign patterns, and certificate coefficients are computed in Python from the fiber data and emitted as Lean. The repository does not claim a complete byte-for-byte generator for every historical source module; the committed statements are checked independently by the kernel, the manifest gate, and the moment-transcription gate.

Rule 4. Only the standard axioms `propext`, `Classical.choice`, and `Quot.sound` occur. In particular, `Lean.ofReduceBool`, the signature of `native_decide`, never appears.

Rule 5. The build is clean: zero errors and zero warnings, with no linter suppressed.

Rule 4 is the load-bearing one. `#print axioms` is transitive, so a single clean line for `no_go_of_moments` certifies the entire dependency cone beneath it.

1.3 The result

No-go orbits certified	51 (every no-go orbit)
Headline `no_go_of_moments` theorems	51
Lines in the orbit modules	41 521
Theorems in the orbit modules	12 521
Orbit-36 rigidity chain	11 modules, 28 412 lines
Ten exact realizations	10 modules, 1 606 lines
Shadow bound, five exact images, three attained	10 modules, 1 487 lines
Lines in the whole development	123 125
Audit entries in `SixQubit/Audit.lean`	507
Audit entries with a non-clean footprint	0
Build errors	0
Build warnings	0
Occurrences of `sorry` in tactic position	0
Occurrences of `native_decide` in tactic position	0

The certified orbits are exactly the fifty-one the manuscript excludes: 5, 6 and 47–52 at projective order three; 12–15, 17–20, 24, 41–44 and 46 at order two; 25–32 and 62–63 at order four; 33–35 and 37–40 at order five; and 53–61 and 64–66 at orders six through nine. Orbit 66 lives in `SixQubit/Order9.lean` for historical reasons and all others in `SixQubit/Orbit<n>.lean`. The remaining fifteen of the sixty-six rank-compatible orbits all target the realized orders $\{2, 3, 5\}$ and are not excluded; four of them (7, 8, 9 and 36) are known to be realized. A per-orbit table, generated from the sources, is in `results/orbit-table.tsv`.

1.4 Toolchain

Lean	`leanprover/lean4:v4.27.0`
Mathlib	`fc278a179e431ed10630af63fc05481680c0c7bd`

The pins are recorded in `lean-toolchain` and `lake-manifest.json`. Nothing outside Mathlib is used mathematically; the other manifest entries are Mathlib's own dependencies.

2 Architecture

The development is layered, and the layering is what made the harder orbits tractable.

2.1 The SS layer

`SS/` (1 608 lines) holds the problem-independent infrastructure: bit strings, Pauli strings, the enumeration of admissible weight-$\leq$ 2 Paulis, and the predicates that decide which Knill–Laflamme conditions are automatic for a given support. Nothing here is orbit-specific.

2.2 Transcription modules

Each orbit has a module containing:

- the two fibers as explicit `def`s with a `decide` proof that they are the residue fibers;
- the `MomentSystem` structure for that orbit, generated from the exact rational moment matrix;
- the derived moment facts, each a theorem over `MomentSystem`;
- `offSum`, `diagBot`, `diagTop`, the incidence/flip/sign lemmas (all by `decide`), and the Walsh expansion lemmas;
- the orbit's Knill–Laflamme relations, each proved by `linear_combination` from expansions with script-computed coefficients;
- the headline `no_go_of_moments`.

2.3 Algebra modules

Thirteen algebra modules carry the mathematics, separated from the bit strings entirely. The first four served the hard orbits of the initial batch:

Module	Serves	Lines
`SixQubit/Block33.lean`	orbit 33	327
`SixQubit/Pair29.lean`	orbit 29	636
`SixQubit/Table31.lean`	orbit 31	490
`SixQubit/Block26.lean`	orbit 26	415

Nine more were added later. Shared layers include `Triangle`, `Frame`, `Spinor`, `ParityFour`, and `Hafnian`. The remaining ones are `Trace5`, `Flip5`, `Kernel5`, and `K5`, the last for the rebuilt orbit 30.

These modules mention no bit strings at all. They take complex numbers and the relations among them and return `False` or an algebraic conclusion. This has a practical consequence worth stating plainly: *the mathematical core of a hard orbit can be written and compiled before any of the transcription plumbing exists*, which is how the risky part gets de-risked first. It also means the algebra can be audited on its own, without wading through several hundred `by decide` lemmas.

2.4 The audit module

`SixQubit/Audit.lean` imports every orbit module and runs `#print axioms` on each headline theorem and on selected intermediate lemmas. Its output, captured in `results/axiom-audit.txt`, is 507 lines, every one reading

```
'SixQubit.<Module>.<theorem>' depends on axioms:
  [propext, Classical.choice, Quot.sound]
```

3 The mechanisms

Thirty-five of the fifty-one orbits fall to one of a small ladder of mechanisms, each of which is driven by a script that finds the certificate and emits the Lean. The ladder, in increasing order of strength:

1. **k-edge Walsh.** A flip mask whose incidence set is small enough that the dressing sign matrix forces one incidence product to vanish, while the moment polytope forces both endpoints to have positive probability.
2. **Support collapse.** A coordinate forced to zero by the polytope removes incidences, which exposes new forced zeros; iterate.
3. **Propagation.** Two-term relations chain across masks.
4. **Quadratic closure.** The squared-modulus consequences of the two-term relations, reduced against the moment system, sometimes linearize and pin the probabilities outright.
5. **Positive row sum.** A shared lemma turning a row of the moment matrix into a contradiction with nonnegativity.
6. **Phase-ratio transitivity.** The quantities $\rho_x = \overline{c_x} d_{\hat{x}}$ are linked across masks; connectivity forces a common phase.
7. **Star hub and mirror permanent.** A hub vertex plus the vanishing of a 2×2 permanent (not determinant — see below).
8. **Case analysis on the active set.** Driven by the polytope's face structure.
9. **Hub without a star.** A vanishing-pair tree replaces the hub.
10. **Diagonal Knill–Laflamme conditions.** The last rung; orbit 25 was the first orbit that could not be closed without them.

A recurring trap deserves its own paragraph, because it caught this work three times. When the fibers admit an involution, a four-term Walsh sum collapses to

$$(1 + \sigma_x\sigma_y)\, P(x, \hat{y}) + (1 + \sigma_z\sigma_w)\, P(z, \hat{w}),$$

which is a genuine cross relation only when *both* sign products are $+1$ and is vacuous otherwise. With the two pairs' relative sign flipped one obtains the 2×2 *determinant* rather than the *permanent*, and the determinant version is satisfied by any rank-one grid, so it yields no contradiction at all. Any argument that reads a permanent where the data supply a determinant is worthless, and the failure is silent.

4 The four hard orbits

Four orbits needed genuinely new arguments. In each case the argument obtained here is substantially shorter than the one in the manuscript, and in two cases it avoids machinery the manuscript needs.

4.1 Orbit 33: a block reflection

$a = (0, 1, 1, 3, 3, 3)$, $q = 5$, fibers $12 + 12$. The first qubit carries angle zero, so both fibers are closed under flipping it and split into six blocks of two; write $c_i, d_i \in \mathbb{C}^2$.

The moment system alone pins every block norm to $R = (3, 2, 2, 1, 1, 1)/10$. The diagonal conditions for F on the zero-angle qubit together with FZ_j on a second qubit give $Cf = JCg$, where C is the 6×6 tail-sign matrix ($\det C = 160$) and $J = \mathrm{diag}(1, -1, \ldots, -1)$; and

$$C^{-1}JC = 2R\mathbf{1}^T - I,$$

which is equivalent to the six matrix identities

$$c_i c_i^\dagger + d_i d_i^\dagger = 2R_i\rho.$$

The cross conditions make blocks $1, 2$ collinear with block 0 on both fibers, which forces $c_j^\dagger d_0 = 0$ and $c_0^\dagger d_j = 0$ for $j = 3, 4, 5$. Orthogonality in $\mathbb{C}^2$ then gives $\rho = I/2$ and simultaneously $\rho = \frac{10}{3} c_0 c_0^\dagger$, so $c_0 c_0^\dagger = \frac{3}{20} I$, whose off-diagonal entry is zero while $\mathrm{normSq}(p_0)\,\mathrm{normSq}(q_0) = 9/400$.

Everything is division free; the only case analysis is `mul_eq_zero` against explicit nonzero rationals.

4.2 Orbit 29: a K_4 of orthogonalities

$a = (1, 1, 1, 1, 2, 2)$, $q = 4$, fibers $16 + 16$, and the two fibers are *not* complementary — a fact that breaks the tooling used elsewhere and required a separate relation extractor (`tools/core/relgen.py`).

Both fibers are closed under X_5X_6, so each splits into eight pairs. Read a pair as a vector in $\mathbb{C}^2$; then every two-term relation is an orthogonality $C_m^\dagger D_n = 0$, and the twelve that survive realise exactly the six edges of K_4 on the four top pairs that carry them.

Two observations shortened the proof considerably relative to the manuscript's fifteen-face audit.

- The uniform identity $P_{\{n,n'\}} = Q_n + Q_{n'} - \frac{1}{4}$ collapses the face table into a single lemma with one hop.
- One never picks a nonzero vector. Instead the common bottom direction is *constructed* from the anchor: $u := (-\overline{v_1}, \overline{v_0})$. It is automatically nonzero, automatically orthogonal to v, and $u \perp D_n$ is literally $\det[D_n, v] = 0$. Only two case splits survive.

4.3 Orbit 31: a 4×4 table

$a = (1, 1, 1, 2, 2, 2)$, $q = 4$, fibers $16 + 16$ and complementary. The last three angles are all 2, so both fibers are stabilised by the Klein four-group on those qubits and split into four cosets of four. The sixteen positions form a 4×4 table with rows A, B_1, B_2, B_3 and columns $e_0, \ldots, e_3$.

The moment normal form — all of it verified exactly against the moment system — is

$$p_1(\bar{v}) = p_0(v) \text{ at every position}, \quad \sum_e \beta_{i,e} = \tfrac{1}{4}, \quad \alpha_e = \sum_i \beta_{i,e} - \tfrac{1}{8}.$$

The first of these lets every active position be written $d_{\bar{v}} = \rho_v \overline{c_v}$ with $|\rho_v| = 1$, and twelve two-term relations make ρ constant along *columns*.

Two facts remove the phase from the Lean side entirely, which is what made this orbit finishable:

- $v_e = \overline{d_{\hat{A}_e}}$: the kernel vector is literally the conjugate of the top amplitude at the complement of A_e. No division, no phase object.
- Two columns share a phase exactly when $v_e a_f = v_f a_e$, a division-free complex equation one can `by_cases` on.

Family one then says the four bottom columns are dependent with coefficients v, so pairing against each column gives *four* equations $\sum_e v_e M_{ef} = 0$, where M_{ef} is the Gram matrix of the B columns. Family three pins $M_{ef} = \overline{v_e} v_f$ whenever the phases differ; the division-free form of that statement, and the fact that closed the last gap, is the unconditional identity

$$\big(M_{ef} - \overline{v_e} v_f\big)\big(v_e \overline{v_f} - a_e \overline{a_f}\big) = 0.$$

Its second factor packages $a_e \neq 0$, $a_f \neq 0$ and $\rho_e \neq \rho_f$ into one side condition. A column alone in its phase class then has bracket $3/8$ and dies at once, and the five branches of the phase partition close in turn.

For comparison, the manuscript closes this orbit by a computer-assisted rational classification of sixteen support types.

4.4 Orbit 26: the cross-anchor contraction

$a = (0, 1, 1, 2, 2, 3)$, $q = 4$, fibers $16 + 16$. Blocks pair into four groups of two; the group $\{6, 7\}$ is the anchor and carries probability $1/8 + 1/8$.

Fix an anchor block j, set $X_i = C_j^\dagger D_i$, and let $R_g = (X_{2g}, X_{2g+1})$. Contracting the outer identity with that anchor gives $R_g J R_h^T = 0$ for the three pairs $g < h$, with J the *symmetric* antidiagonal form; three nonzero pairwise J-orthogonal rows in $\mathbb{C}^{1\times 2}$ lie on one coordinate axis.

The axis, however, is chosen *per anchor*, and an earlier version of this argument silently used both anchors at once. That is a real gap: an adversarial review produced explicit witnesses in which $C_6 \not\parallel C_7$ and the counting closes with slack. The repair is to contract the outer identity with *different* anchors in the two slots, on the a index with D_7 and on the b index with $\overline{C_6}$, which yields

$$R_g^{(7)} J R_h^{(6)T} + R_h^{(7)} J R_g^{(6)T} = 0,$$

reading $a_g b_h + a_h b_g = 0$ in the mixed configuration — whose only solution has one of the six entries zero. Two caveats, both checked rather than assumed: the cross relation is *vacuous* in the aligned branch, which is closed separately by a moment and Cauchy–Schwarz argument; and the fact

$$P_0 + P_2 + P_4 = P_1 + P_3 + P_5 = \tfrac{3}{8}$$

is load-bearing — an ablation shows that dropping it makes the whole fact set feasible again, precisely at the aligned point $P = (0, \frac{1}{4}, 0, \frac{1}{4}, 0, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$.

5 Orbit 36: rigidity rather than nonexistence

Fifty-one of the sixty-six moment orbits are excluded. Orbit 36 is not: it is realized, by an exact $((6, 2, 3))$ code whose projective diagonal group is C_5. What is certified about it is therefore a different kind of statement, and a stronger one than a no-go. From the same raw material every no-go module consumes—support conditions, the moment system, and Knill–Laflamme rows of Pauli weight at most two—the code is pinned uniquely.

5.1 The statement

`SixQubit/Orbit36Final.lean` carries

`orbit36_rigidity` : (2 support conditions) $\wedge$ (1 moment system) $\wedge$ (22 KL rows) $\Longrightarrow$ $\mathcal{R}$,

where the conclusion $\mathcal{R}$ has three parts:

1. *the magnitudes are forced*: the twelve bottom probabilities are $1/10$ on six states and $1/15$ on the other six, and likewise for the top fiber—no state of either fiber may vanish;
2. *the code is unique*: `CodeEquiv.Equivalent` relates the pair (c, d) to the explicit canonical pair (c_{36}, d_{36});
3. *the weight enumerator is determined*: $A = \big(1,\, 0,\, \frac{21}{25},\, 0,\, \frac{291}{25},\, \frac{384}{25},\, \frac{79}{25}\big)$.

The hypotheses matter as much as the conclusion. They are exactly what an exact distance-three code supplies and nothing else: no amplitude is assumed nonzero, no phase is assumed generic, no basis is assumed chosen. The support conditions are containments, $\forall s \notin S_0,\ c\,s = 0$, so zero amplitudes inside the fibers are permitted throughout—which is what makes the full-support conclusion in part (1) a genuine consequence rather than a restatement of an assumption.

5.2 What the equivalence means

Part (2) is stated with respect to `CodeEquiv.LocalMonomial`, a qubit permutation together with a bit flip and a diagonal (α_i, β_i) per qubit with $|\alpha_i| = |\beta_i| = 1$, composed with a unit scalar on each logical state and an optional exchange of the two logical states. This is *smaller* than local-unitary equivalence, so proving uniqueness at this level is the stronger claim; the manuscript states the result up to local unitaries and physical-qubit permutations and notes the proof achieves the smaller relation, which is the one formalized here.

5.3 The chain

Eleven modules, 28 412 lines and 1 894 theorems:

`K4Kernel`	the K_4 kernel used by the phase argument
`Orbit36Code`	the canonical pair, its fibers, and `canonical_is_code`
`Orbit36`	fibers, moment system, `magnitudes_forced`, `full_support`
`Orbit36Phase`	$A_1 = 0$, the triangle sum, the weight-two table
`Orbit36Enum`	the enumerator machinery through weight three
`CodeEquiv`	local-monomial equivalence and its invariance lemmas
`Orbit36Class`	the classification up to that equivalence
`Orbit36Enum4`	weights four to six
`MacWilliams`	the subset MacWilliams identity at $n = 6$
`Orbit36Final`	the headline, assembled
`Orbit36Vacuity`	the non-vacuity witness

5.4 Non-vacuity

A rigidity theorem with unsatisfiable hypotheses is worthless in a way a no-go is not: a no-go with unsatisfiable hypotheses is merely uninformative, whereas a rigidity statement would be asserting properties of nothing. `Orbit36Vacuity.rigidity_nonvacuous` applies the headline to the explicit canonical pair, discharging all twenty-five hypotheses. The moment hypotheses

are discharged by rewriting to the canonical probabilities and the KL rows by routing through `Orbit36Code.canonical_is_code`, so the witness is the code itself rather than a degenerate point mass—which is what the corresponding witnesses in `SixQubit/Vacuity.lean` are for the no-gos, and correctly so, since there a point mass suffices to show the hypotheses are not self-contradictory.

5.5 Orbit 36 at the level of codes

Every orbit-36 result in Section 5—the forced magnitudes, the local-monomial uniqueness, the enumerator—took its moment system and its twenty-two Knill–Laflamme rows as *hypotheses*. The fifty-one no-go modules never did: each `CodeOrbit`n`.no_code` derives them from the single statement that $![c, d]$ is a normalized distance-three code supported on the two fibers. Orbit 36 was the one module in the development without that bridge.

The difference is not presentational. Before it, the rigidity theorem was a statement about amplitude pairs satisfying certain moment conditions; the step from "is an exact code in orbit 36" to "satisfies those conditions" was true but unproved. Only with the bridge does the theorem say what the manuscript says.

It was found by asking a mechanical question of the whole tree: for each of the eleven realized orbits, which theorems exist, what are their hypotheses, and do those hypotheses include `SS.HasDistance`? Scanning for that turned up three occurrences of `HasDistance` across the orbit-36 chain, all three in a *conclusion* and none in a hypothesis.

`tools/generators/gencode36.py` closes it by the route `gencode9.py` used for orbit 66, whose fiber structure is the same. Two differences: orbit 36 names its top states $z_0, \ldots, z_{11}$ rather than complements, and the generator checks $z_i = \overline{y_i}$ before writing anything; and its moment structure carries nonnegativity fields that orbit 66's does not, supplied by $\|\cdot\|^2 \geq 0$. The conclusion of `rigid_code` is lifted verbatim out of `orbit36_rigidity` rather than restated, so the wrapper cannot claim more or less than the theorem it feeds.

5.6 The gate that did not exist

The four independent checks of Section 9.3 all key off the `no_go_of_moments` headline and obtain their module list from the reviewed 51-module inventory. Orbit 36 has no such headline and is in no such inventory, so every one of them silently skipped it: the single largest body of Lean in this repository was the only part with no transcription gate at all.

`tools/verification/rigidity.py` closes that. It checks the same obligations—orbit identity, fibers, no smuggled hypothesis, Pauli weight, moment-row content, the order in which the headline passes probabilities, and that the non-vacuity witness really applies the headline—in twenty-one assertions.

Two details are worth recording, because both are the kind of thing that makes a checker agree with a wrong module.

First, the orbit identity is not a string comparison. The Lean modules use the angle record $a = (1, 1, 1, 1, 2, 3)$ at $q = 5$ with fibers at residues 0 and 4; `tools/manifest.json` records orbit 36 as $a = (1, 1, 1, 2, 2, 4)$ with $\delta = 1$. Those are different vectors, and that they describe the same orbit is a fact to be proved, not a discrepancy to be waved away. The checker reduces both to a canonical form under the group generated by local flips ($a_j \mapsto -a_j$, under which both fibers shift together so δ survives), qubit permutation, unit scaling ($a \mapsto ua$, $\delta \mapsto u\delta$) and exchange of the two logical states ($\delta \mapsto -\delta$). Both reduce to $\big((1, 1, 1, 1, 2, 2),\ \delta = 1\big)$.

Second, the moment rows are re-derived here from the fibers rather than shared with `momentgate.py`. Two checkers built on one parser fail together; that already happened once in this repository, when a declared-row counter and a parsed-row counter shared a fragile prefix, missed the same row, and agreed.

The mutation suite carries fourteen corruptions and initially caught only two of them. The reason is instructive and is the same failure again in miniature: an anchor such as `(ho0 : offSum c d ex1 w0 = 0)` occurs five times in `Orbit36Final.lean`—once in the headline and again in the lemmas it is assembled from—so a whole-file replacement mutated a lemma and left the headline pristine, and twelve mutations were testing nothing while reporting success. Mutations are now applied inside the headline region only, and a mutation that fails to inject is reported as inert rather than as passing.

6 Ten realizations

Of the sixty-six moment orbits, fifty-one are excluded and fifteen are realized. Eleven of the fifteen have a known exact weight-enumerator image; orbit 36 is one of them and is treated in Section 5. This section covers the other ten—orbits 7, 8, 9, 10, 11, 16, 21, 22, 23 and 45—at the level of *existence*: each carries an explicit pair of words and a kernel proof that the pair is an exact $((6, 2, 3))$ code, so the orbit is nonempty.

6.1 One layer, ten tables

The orbit-36 code module carried its own ring of algebraic integers, its own embedding into $\mathbb{C}$, and its own transport from a decided certificate to genuine Knill–Laflamme matrix elements. None of that depends on the amplitude table, so it is collected in `SixQubit/Realize.lean` and the ten modules supply only data: two support lists, two amplitude tables, and a scale.

The ring is $\mathbb{Z}[\sqrt{2}, \sqrt{3}, i]$, carried as eight integers. It is small enough to decide with and still large enough for every realization here, because each table is recorded *up to a single real scale*:

words	scale	entries
sixteen real terms	$1/4$	± 1
eight real terms	$1/4$	$\pm\sqrt{2}$
eleven complex terms	$1/12$	$2\sqrt{3}\,\zeta_6^k$, $2\sqrt{6}\,\zeta_6^k$
twelve complex terms (orbit 36)	$1/(2\sqrt{30})$	$2\sqrt{3}\,\zeta_6^k$, $2\sqrt{2}\,\zeta_6^k$

Pulling the irrational part out as a scale is what keeps the ring small. Orbit 36's amplitudes are $1/\sqrt{10}$ and $1/\sqrt{15}$, which would need $\sqrt{5}$; written as $\sqrt{3}/\sqrt{30}$ and $\sqrt{2}/\sqrt{30}$ they do not. For the ten modules here the scales are rational, so the normalization $\overline{s}\, s\, N = 1$ is closed by `norm_num`.

The certificate the shared layer consumes is smaller than orbit 36's. Being a code requires exactly three things of every Pauli of weight at most two: the two diagonal elements agree and both off-diagonal elements vanish. Which Paulis carry a nonzero diagonal scalar is extra information—orbit 36 needs it for the enumerator, these modules do not—so it is not part of `checkKL`. The scale is a free parameter of the transport theorem: it cancels from the off-diagonal equations and enters the diagonal one symmetrically.

6.2 Two independent computations of the same certificate

`tools/generators/genreal.py` writes all ten modules. Two habits are deliberate.

First, the amplitudes and the orbit records are *parsed* from the source tables, never transcribed. Hand transcription is where this project's errors have actually come from, and the same script that parses the base words also applies the qubit permutation, the translation and the optional exchange of the two logical words that place each base into its orbit.

Second, the generator re-checks the whole certificate—all 4096 mask pairs, filtered by weight—in Python, in the same eight-integer arithmetic the Lean uses, and refuses to write

a table that does not verify. The kernel `decide` is then a second, independent confirmation rather than the only one. Both computations agree on all ten.

6.3 What the kernel cannot check

Each module decides its own certificate, so the kernel establishes that the exhibited pair is a code. It cannot establish that the pair belongs to the orbit the module is named for: `tools/manifest.json` is not a Lean object. That is exactly the gap that let four no-go modules target the wrong residue before `gate.py` existed.

`tools/verification/realized.py` closes it, with twelve assertions per module. The orbit identity is compared by canonical form under the group generated by local flips, qubit permutation, unit scaling and exchange of the two logical words, not by string equality—equivalent angle records look different, and orbit 36 is the standing example. Both fibers are recomputed from (a, q), the two supports are checked disjoint, and the amplitude norms are re-derived in the same ring. Its `--selftest` injects six corruptions and requires every one to be caught.

6.4 A cross-check worth recording

The exact weight enumerators of the ten exhibited codes were computed independently, in exact arithmetic, and compared with the images the manuscript claims:

orbits	(A_1, A_2, A_3)	manuscript
8, 11, 23	$(1, 0, 0)$	$\mathcal{E}_1^A = (1, 1, 0, 0, 15, 15, 0)$
7, 9, 10, 16, 21, 22, 45	$(0, 1, 0)$	$\mathcal{E}_2^A = (1, 0, 1, 0, 11, 16, 3)$

Every one agrees, and every enumerator sums to 32. This is independent evidence for both sides: the generated codes are the ones the manuscript intends, and the manuscript's table is right about them.

6.5 Five exact images

For five of the ten—orbits 10, 11, 16, 22 and 23—the certification goes past existence to the exact weight-enumerator image, which is the manuscript's actual theorem for them. The argument is a squeeze, and every step of it is finite.

A projective-order-two orbit with $q = 4$ has a diagonal gate acting logically as $\mathrm{diag}(1, -1)$, so its square is scalar on any code in the orbit. That square is the Z-string on the coordinates where a is odd, and its sign on a support word x is $(-1)^{A_a(x)}$, which is $+1$ because $A_a(x)$ is 0 or 2 modulo four on the two fibers. So the string fixes the code—a condition over the two supports, decided by the kernel.

A Pauli fixing both logical states has projector trace 2, so it contributes 4 to the coefficient at its own weight, and coefficients are sums of squared norms, so that is a lower bound. The shadow inequality is the matching upper bound on the sum of the first three. Nothing is left free: $\mathrm{Aco}_w = 4$ and the other two vanish. The six-qubit MacWilliams normal form then determines every remaining coefficient.

`SixQubit/Pin.lean` carries the squeeze; each generated `Orbit`n`Enum.lean` supplies its mask and applies it. The conclusion is about *every* code supported in those fibers, not only the exhibited one, and the realization module supplies a code attaining it, so the image is a single point:

$(1, 1, 0, 0, 15, 15, 0)$ for orbits 11 and 23, $(1, 0, 1, 0, 11, 16, 3)$ for orbits 10, 16 and 22.

Orbit 21's forced string has weight four and orbit 45 has $q = 6$, where the square is not a Pauli at all; orbits 7, 8 and 9 have $m = q = 3$, where the cube is the identity and carries no information. Those five keep only their realizations here.

6.6 Three more enumerators, and the line the mask draws

Orbits 7, 8 and 9 also carry a Pauli of weight at most two with projector trace of modulus two, so the same squeeze fixes the enumerator of each exhibited code: Z on qubits $\{0,2\}$ for orbit 7, on $\{4\}$ for orbit 8, on $\{1,2\}$ for orbit 9. The first two act by -1 rather than $+1$, which costs nothing—only $|\operatorname{Tr}(P\Pi)|^2$ enters, and $(-2)^2 = 2^2$—but it did cost a search: looking only for $\operatorname{Tr}(P\Pi) = +2$ finds nothing for orbits 7 and 8, and the right condition is modulus one. The sign is exactly the parity of the mask's overlap with the support, which is uniform across it.

What separates these three from orbits 10, 11, 16, 22 and 23 is where that parity is uniform. There it holds on the whole *fiber*, so the conclusion is about every code supported in the orbit. Here it holds only on the exhibited eleven-word support, and a search over all sixty-three nonzero masks finds none that is uniform fiber-wide. So orbits 7, 8 and 9 get attainment of one point and nothing more; the manuscript reaches further by reducing the support first, which is not attempted here.

That distinction is worth stating plainly because it is easy to lose. The hypothesis $\forall s \notin L_0,\ c\,s = 0$ reads like "supported in the orbit" but says "supported in this list". The two coincide exactly when the list is the fiber, and `L0_eq_fiber` now decides that where it is true rather than leaving it to a reader's assumption.

6.7 The shadow inequality, and what is still missing

Existence is not the whole claim. For eight of these orbits the manuscript asserts a *unique* enumerator and for three an exact interval, and both directions of those statements need more than one code. The upper bound in every case runs through the zeroth Rains shadow inequality $A_1 + A_2 + A_3 \le 1$. That inequality is now proved.

Its usual derivation builds the spin flip $\Pi \mapsto Y^{\otimes n}\Pi^{\mathsf{T}}Y^{\otimes n}$ and quotes positivity of $\operatorname{Tr}(\Pi\widetilde{\Pi})$. There is a shorter route. Summing the alternating Pauli sum over the Z-mask first collapses it by the character sum already in `SixQubit/MacWilliams.lean`, and what survives factors:

$$\sum_{P}(-1)^{\mathrm{wt}\,P}\big|\operatorname{Tr}(P\Pi)\big|^2 \;=\; 2^n \sum_{a,b}\Big|\sum_{u}(-1)^{|u|}\psi_a(u)\,\psi_b(\bar{u})\Big|^2,$$

a sum of squared norms, so nonnegativity is immediate and no operator, transpose or matrix positivity appears. The identity was checked numerically before any Lean was written—on the orbit-36 code it gives $128/25$, exactly what the six-qubit MacWilliams normal form predicts, and it holds on random non-code pairs too, confirming that it is an algebraic identity rather than a consequence of the code property. `SixQubit/Shadow.lean` proves it. Both sides are computed down to a common quantity: one direction is an interchange of summation at depth four, the other does the Z-sum first, and the surviving index is a complement. The statement `alt_sum_nonneg` holds for every n and every number of logical states; specializing to $n = 6$ and grouping by weight gives `shadow_bound`, namely $A_1 + A_2 + A_3 \le 1$ for an exact $((6, 2, d \ge 3))$ code, in the unnormalized form $\mathrm{Aco}_1 + \mathrm{Aco}_2 + \mathrm{Aco}_3 \le 4$ that the six-qubit MacWilliams normal form already in the development consumes.

What still separates this from the manuscript's exact-image theorems is the orbit-specific half of each: a lower bound forcing $A_1 + A_2 \ge 1$ or its analogue, and for orbits 7, 8 and 45 a one-parameter family attaining every value of a continuum. Neither is finite in the way everything else here has been.

7 A tooling bug that gated three orbits

The scan tooling under-generated the admissible Pauli set. For a weight-one flip e it returned only $\{0\}\cup\{e_j\}$, seven strings, where the correct set is $\{0, e_i\}\times\{0, e_j\}$, twelve; it thereby dropped

every Y_iZ_j. For $e = 0$ it returned nothing at all, dropping all twenty-two pure-Z conditions. The correct total over all $|e| \le 2$ is

$$154 = 1 + 6 \cdot 3 + 15 \cdot 9,$$

the number of weight-≤ 2 Pauli operators on six qubits, which is the check that should have been run at the start.

Three of the last four orbits were blocked by this. Orbit 33's argument needs $F \otimes Z_j$ for F ranging over I, X, Y, Z on the zero-angle qubit, and three of those four families were missing.

No earlier certification was invalidated by this under-generation bug. Those proofs used a *subset* of the true conditions, which makes them harder, never wrong. The fix lives in `tools/core/kl.py`, which enumerates all sixty-four dressings and filters on $|\operatorname{supp}(e) \cup \operatorname{supp}(z)| \le 2$ rather than being clever.

8 Errors made, and how they were caught

Five reasoning errors were made during this work and all five were caught before anything unsound entered the development. They share one shape: *asserting that a relation has a certain form without checking that form against the originals.* Recording them, and the detection methods, is more useful than pretending the path was straight.

1. **A wrong δ.** Choosing the second fiber by size alone made the search accept easy certificates for the wrong mathematical objects. A later d_{LP} cross-check happened to disambiguate some orbits then still outstanding, but it is not the definition of δ and did not revisit modules already marked complete. Orbits 30, 53, 55 and 65 therefore used residues of additive order two instead of the orders their projective logical gates demand. A collaborator's audit found this; all four were rebuilt with the defining value $\delta = q/m$, and the corrected fibers are disjoint from the old ones. The guard that should have existed from the start is now `tools/manifest.json` plus `tools/verification/gate.py`, which checks the paper parameters, residue and both support fibers. The separate `tools/verification/momentgate.py` parses exact rational linear forms and checks every declared moment row against the probability order used by its headline theorem.

2. **The collapse trap, three times.** A four-term sum was read as a cross relation when the sign products made it vacuous, or as a permanent when the data gave a determinant. Fixed by writing an extractor (`tools/core/relations.py`) that computes every two-term relation genuinely in a mask's row space.

3. **Orbit 26: per-edge matrix relations that do not exist.** The weight-one masks carry eight incidences with rank only four and admit *no* two-term relation; only group-level sums exist.

4. **Orbit 31: a family-three statement pushed too strong.** Claiming each term vanishes separately gives $G_{ef}t_{ef} = 0$, which is false. The bottom sign is $(-1)^{z\cdot x}$ and the A row sits at complementary last-three-bits from the B rows, so an odd-weight dressing flips the A-row sign relative to the others.

5. **Orbit 31: a covering count with a counterexample.** The claim that covering six column pairs with three rows forces one nonzero entry per row is false. For example,

   $$Z_1 = \{0, 1, 2\}, \qquad Z_2 = \{0, 1, 3\}, \qquad Z_3 = \{2, 3\}$$

   covers all six pairs with empty intersection. The branch does close, but by the full permanent condition, not by counting.

The detection methods, in increasing order of usefulness:

1. **Row-space membership.** Reduced row echelon form against the mask's dressing sign matrix answers "is this relation actually available?" directly. This caught error 3.

2. **Replay against the originals.** Take a solution of the reduced system and evaluate the *raw* Knill–Laflamme conditions on it. Exactly two failed for error 4, and both were weight-one dressings, which pointed straight at the sign. This is cheap and should be run on every reduction.

3. **Adversarial review.** A multi-agent pass whose agents were instructed to *refute* rather than confirm produced explicit witnesses for error 3's consequence in orbit 26, including a configuration satisfying every fact the argument used.

4. **Writing the Lean.** Error 5 surfaced while formalising the branch: the combinatorial step had to be made concrete, and the counterexample fell out. This is the cheapest of the four and argues for a working practice: *do not derive at length on paper and formalise in one batch; formalise as you go.*

For the final two orbits the work was delegated to multi-agent workflows under three non-negotiable instructions: verify by machine rather than assert, actually compile rather than report compiling, and classify every hypothesis of the headline theorem. That last one is the important one. A headline `False` theorem is only as good as its hypotheses, and a proof becomes a shell the moment an algebraic fact that should have been derived is smuggled in as an assumption — a defect invisible from the outside. Both final headlines were classified by hand after the fact:

Orbit	Hypotheses	`offSum = 0`	`diagBot = diagTop`	`MomentSystem`	Other
31	46	33	12	1	0
26	41	24	16	1	0

9 Reproduction

9.1 Checking the proofs

From the repository root:

```
lake build
```

should report zero errors and zero warnings. Then

```
lake build SixQubit.Audit
```

prints the 507 axiom lines; every one must read `[propext, Classical.choice, Quot.sound]`. A stored copy of the expected output is `results/axiom-audit.txt`, so

```
lake build SixQubit.Audit 2>&1 | grep "depends on axioms" \
  | sed 's/^info: [^ ]* //' | diff - results/axiom-audit.txt
```

should produce no output.

To confirm the absence of the forbidden constructs:

```
grep -rn "sorry\|native_decide" SixQubit/ SS/
```

Every hit must be inside a docstring or comment. At the recorded revision there are 181 occurrences on 123 lines; most are the sentence stating that `native_decide` is not used. The transitive `#print axioms` audit is the load-bearing check: an active escape hatch would add `sorryAx` or `Lean.ofReduceBool`, neither of which appears.

9.2 Regenerating the Lean

Most transcription modules were emitted or repaired by scripts in `tools/generators/`. Their inputs vary: some read `tools/orbits/`, some read `tools/manifest.json` or checked certificate data, and some algebra layers are hand-written. The provenance index in `tools/README.md` names the major routes and explicitly marks the historical order-nine module, for which no complete raw-module generator is committed. The scripts require Python with `sympy`, `numpy` and `scipy`. For scripts using the shared modules, set `PYTHONPATH` to include `tools/core` and `tools/orbits`, or run from the repository root with

```
python -c "import sys; sys.path[:0] = ['tools/core','tools/orbits']; ..."
```

9.3 Re-running the independent checks

`tools/verification/` holds the scripts that answer questions the Lean proofs do not: whether an orbit is excluded numerically at all, which relations are genuinely available, and which branches of a case analysis are live. Highlights:

Script	What it answers
`feas2.py`, `feasnew.py`	Is the full weight-≤ 2 system numerically feasible or infeasible, including positive and negative controls?
`gate.py`	Do the paper parameters, residue and two supports match?
`momentgate.py`	Is every declared moment row the exact intended linear form?
`momentgate.py --selftest`	Does the non-exhaustive mutation suite fail closed?
`classify.py`	Does every algebraic headline hypothesis have an allowed form?
`admis.py`	Does every headline KL row have Pauli weight at most two?
`close31.py`	Orbit 31 against the raw conditions, phases free
`branch31.py`	Which phase partitions of orbit 31 are excluded
`collapse29.py`	Orbit 29's support collapse
`stepA\_*.py`	The four silent failure modes behind orbit 26's Step A
`t26\_*.py`, `x26\_*.py`	Orbit 26 facts, ablations, branch tree

`feas2.py` includes certified orbits as controls: an orbit with a kernel-checked no-go proof must come out excluded, and it does (orbit 25 gives a best residual of 0.79). Running it without the controls would not be a test.

9.4 The workflow records

`results/workflows/` holds the scripts of the multi-agent passes and their verdicts, including the adversarial review that found the orbit 26 gap and the audits of the final two assemblies. They are kept because they contain the negative results — which relations are *not* available, which branches are vacuous — that are expensive to rediscover and easy to assume away.

10 Repository layout

Path	Contents
`SixQubit/`	the fifty-one orbit modules, thirteen algebra modules, the audit
`SS/`	problem-independent bit-string and Pauli infrastructure
`formalization/`	this process document in source and PDF form
`tools/core/`	fibers, Pauli enumeration, moment matrices, relation extractors
`tools/orbits/`	per-orbit data and block structure
`tools/generators/`	the scripts that emit the Lean modules
`tools/verification/`	numerical and symbolic checks independent of Lean
`results/`	axiom audit, orbit table, workflow records

11 Open items

One item concerns the formalization itself.

1. **The classification is not formalized.** Neither the reduction of an arbitrary code to one of the sixty-six rank-compatible moment orbits, nor the equivalence of other angle records to the chosen representatives, has a Lean proof.

Two further items concern the manuscript rather than the formalization.

1. Whether the equations and probability data developed here should be folded back into the manuscript. The arguments obtained for orbits 26, 29, 31 and 33 are shorter than the manuscript's and in two cases avoid machinery it uses (matrix inverses and a genericity argument for orbit 26; a sixteen-type support classification for orbit 31), so there is a case for replacing rather than supplementing.

2. In a sibling companion manuscript, `../A_dyadic_ladder/paper/main.tex`: the sentence at line 1045 asserting that no lower bound is known is not correct for stabilizer codes, and the table caption at line 1000 needs revision. Neither touches the present development.